claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-aafa0f8a7cc7c6d61.jsonl`
- SHA-256: `9b534233ea37cc3e09838a3302d277d1486cf9fbb2c2f5be78fef82def30e386`
- Source modified: `2026-06-20T09:03:04+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`

Transcript

1. user (2026-06-20T09:00:58.222Z)

```
Adversarially verify this review finding against the ACTUAL code in the working tree. Default to
'false-positive' unless you can point to the concrete line(s) and a real failure scenario.
Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off
origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the
worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py  (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py           (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py   (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py         (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py       (_build_native_runner override; WASB
substrate only)
- backend/config/models.py                           (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py                         (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

FINDING [worker-extract] (major) @ backend/worker/__main__.py:183 (_resolve_train_detector) ?
runner, _ = seg._resolve_runner(config.get("indexing", {})): _resolve_train_detector does not
wrap seg._resolve_runner in try/except, but _resolve_runner -> _build_native_runner RAISES
NotImplementedError for any family without a native runner (tracknet_hybrid, or fusion_hybrid
with substrate='tracknet'). So `worker train --trajectory-source detector --segmenter
```

tracknet_hybrid` with detector_impl='native' crashes cmd_train with an uncaught traceback instead of returning a clean exit code 2.
WHY: Reproduced directly: calling _resolve_train_detector with segmenter='tracknet_hybrid' + indexing.detector_impl='native' raises `NotImplementedError: detector_impl='native' is not supported for the 'TrackNet' family`. The function's own contract (docstring: 'Returns the runner, or prints an error + returns None') is violated; the worker dies on a normal misconfiguration. The seam already produces clean errors for non-trajectory segmenters and failed healthchecks, but not for the build step it calls.
PROPOSED FIX: Wrap the resolve in try/except: `try: runner, _ = seg._resolve_runner(config.get('indexing', {})) except NotImplementedError as e: print(f'[worker] detector not available: {e}'); return None`. Keeps cmd_train returning 2 on misconfig instead of a traceback.

2. assistant (2026-06-20T09:01:00.537Z)

I'll verify this finding against the actual code in the working tree. Let me start by reading the relevant files.

3. user (2026-06-20T09:01:01.026Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import os
22      import sys
23      import tempfile
24      from typing import Any, List
25
26      from backend import storage
27      from backend.config import load_config
28      from backend.infrastructure.database import Database
29      from backend.pipeline.segmenters.base import segmenter_registry
30      from backend.pipeline.validators.base import registry as validator_registry
31      from backend.reporting import emit_indexer_report
32      from backend.results import Failure
33      from backend.utils.hashing import compute_video_id
34
35
36      def _coerce(v: str) -> Any:
37          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
38          low = v.lower()
39          if low in ("true", "false"):
40              return low == "true"
```

```
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

```

```
        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0
```

```
160
161
162     def _resolve_train_detector(args: argparse.Namespace, config: Any):
163         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165         stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169         no shuttle detector and is rejected.
170         """
171         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173         seg_cls = segmenter_registry.get(args.segmenter)
174         if not seg_cls:
175             print(f"[worker] unknown segmenter: {args.segmenter!r} "
176                   f"(have {list(segmenter_registry.list_available())})")
177             return None
178         seg = seg_cls(None, config)
179         if not isinstance(seg, TrajectoryHybridSegmenter):
180             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182             return None
183         runner, _ = seg._resolve_runner(config.get("indexing", {}))
184         ok, msg = runner.healthcheck()
185         if not ok:
186             print(f"[worker] detector environment not ready: {msg}")
187             return None
188         print(f"[worker] detector ready ({args.segmenter}): {msg}")
189         return runner
190
191
192     def cmd_train(args: argparse.Namespace) -> int:
193         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196         Two trajectory sources (the train pipeline + harness are identical either way):
197           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201             is the M3.5 "first real training" path; only the trajectory source flips.
202         """
203         import subprocess
204
205         config = load_config(args.config) if args.config else load_config()
206         _apply_overrides(config, args.set)
207         storage_cfg = config.storage.model_dump()
208
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210         labels_dir = os.path.join(scratch, "labels")
211         traj_dir = os.path.join(scratch, "trajectories")
212         videos_dir = os.path.join(scratch, "videos")
213         os.makedirs(labels_dir, exist_ok=True)
214         os.makedirs(traj_dir, exist_ok=True)
215
```

```
        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
```

```
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
```

```
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

4. assistant (2026-06-20T09:01:03.569Z)

Now let me look at the `_resolve_runner` and `_build_native_runner` methods in the segmenters to verify the claim that they raise `NotImplementedError`.

5. user (2026-06-20T09:01:04.270Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
```

```
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55              raise NotImplementedError
56
57          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58              """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59              native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60              raise NotImplementedError(
61                  f"detector_impl='native' is not supported for the "
62                  f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
```

```
            if impl == "stub":
                from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
                logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                            self.display_label or "trajectory")
                return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
            if impl in ("native", "native_wasb", "wasb_native"):
                logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
                            self.display_label or "trajectory")
                return self._build_native_runner(idx_cfg)
            return self._build_runner(idx_cfg)

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
```

```
135             return list(range(len(windows)))
136
137         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
138                           player_pool: Optional[List[str]] = None,
139                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140             # override-ignore: the ABC declares the Result contract (Success|Failure)
141             # but every live segmenter still returns a bare list ? the documented
142             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
143             label = self.display_label
144             # --- Sport profile + AI provider wiring (identical across segmenters) ---
145             sport_name = self.config.get("sport", "badminton")
146             try:
147                 sport_profile = sport_registry.get(sport_name)
148             except KeyError as e:
149                 logger.error(str(e))
150                 return []
151
152             idx_cfg = self.config.get("indexing", {})
153             # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154             # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
155             # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156             # measure it against the Gemini path, or to avoid the cloud entirely). With
157             # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158             skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160             if skip_ai:
161                 model_name = "local-noai"
162                 logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                             "be emitted as rallies; no %s handoff, no API key needed.",
164                             label, provider_name)
165             else:
166                 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
168                 api_key = os.environ.get(api_key_env_var)
169                 if not api_key:
170                     from backend.pipeline.segmenters._keyhint import api_key_hint
171                     logger.error(
172                         f"{api_key_env_var} environment variable is not set! "
173                         f"To run the {provider_name} handoff, set your API key. "
174                         + api_key_hint(api_key_env_var)
175                     )
176                     return []
177                 try:
178                     provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179                 except KeyError as e:
180                     logger.error(str(e))
181                     return []
182
183             video_duration = get_video_duration(video_path)
184             if video_duration <= 0:
185                 logger.error("Invalid video duration.")
186                 return []
187             fps, frame_width = self._probe_fps_width(video_path)
188
189             # --- Runner config + windowing thresholds ---
190             # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191             # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
```

```
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            runner, runner_params = self._resolve_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                return []
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

```

```
            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
```

```
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
```

```
                        segments_data.append({
                            "id": segment_id, "start_time": start, "end_time": end,
                            "duration": dur, "metadata": metadata,
                        })
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)

            return segments_data

```

6. assistant (2026-06-20T09:01:08.239Z)

I've confirmed the key facts. Let me verify the `_resolve_train_detector` call path. The finding says line 183 calls `seg._resolve_runner(config.get("indexing", {}))`. But I notice at line 178, `seg_cls(None, config)` is called and then at line 183, `config.get("indexing", {})` is called on `config`. Let me check what type `config` is ? whether it's a dict or a Pydantic config object, since the call uses `.get()`.

7. user (2026-06-20T09:01:08.945Z)

```
"""Configuration package.

Public API:
    from backend.config import load_config, AppConfig, Config

This package is the canonical place for all configuration access.
"""

from .loader import load_config, save_default_config
from .models import (  # re-export common ones
    AppConfig,
    Config,
    IndexingConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)

__all__ = [
    "load_config",
    "save_default_config",
    "AppConfig",
    "Config",
    "IndexingConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
]

```

8. user (2026-06-20T09:01:11.158Z)

```
"""Configuration loader and saver.

Provides a single place to load validated AppConfig with built-in defaults
merged with optional file overrides (config.json).

This replaces scattered json.load + .get() patterns throughout the codebase.
"""

from __future__ import annotations

import json
import logging
```

```
    from pathlib import Path
    from typing import Any

    from pydantic import BaseModel

    from .models import AppConfig

    logger = logging.getLogger(__name__)

    DEFAULT_CONFIG_PATH = Path("config.json")


    def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

```

```
75          # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
76          # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
77          # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
78          # loader's "bad input is non-fatal" contract.
79          if not isinstance(file_data, dict):
80              logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
81                             cfg_path, type(file_data).__name__)
82              file_data = {}
83
84          if file_data:
85              unknown = _unknown_key_paths(file_data, AppConfig)
86              if unknown:
87                  logger.warning(
88                      "[CONFIG WARNING] %s contains keys the typed config does not "
89                      "recognize (typo'd or removed knobs ? top-level extras are kept "
90                      "but unread; unknown section keys are silently dropped): %s",
91                      cfg_path, ", ".join(sorted(unknown)),
92                  )
93
94          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
95          merged = {**_get_builtin_defaults(), **file_data}
96
97          return AppConfig.model_validate(merged)
98
99
100     def save_default_config(path: str | Path | None = None) -> Path:
101         """Write a fresh default config.json.
102
103         Used by scripts/doctor.py diagnostics and the --setup CLI path.
104         Overwrites if the file exists.
105         """
106         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
107         cfg = AppConfig()
108         cfg_path.parent.mkdir(parents=True, exist_ok=True)
109         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
110         return cfg_path
111
112
113     # Backwards-compatible alias used by some older call sites during transition.
114     def get_default_config() -> dict[str, Any]:
115         """Return defaults as a plain dict (for transitional use only)."""
116         return _get_builtin_defaults()
117
```

9. assistant (2026-06-20T09:01:15.616Z)

So `load_config` returns an `AppConfig` Pydantic model. But `_resolve_train_detector` calls `config.get("indexing", {})` at line 183. A Pydantic `AppConfig` model wouldn't have a `.get()` method by default. Let me check the `AppConfig` model to understand how `config.get(...)` works, since the segmenter is constructed with `seg_cls(None, config)` and then `seg._resolve_runner(config.get("indexing", {}))` is called.

10. user (2026-06-20T09:01:16.234Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
```

```
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
```

```
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
74          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
81          # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
82          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
83          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
84          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
85          stream_video: bool = False
86
87
88      class FusionConfig(BaseModel):
89          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
90
91          Every default reproduces control behaviour: the fusion segmenter persists the
92          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
93          true ? the person-cue features, but it does NOT gate the served handoff unless a
94          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
95          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
96          supplied ? off-court persons (spectators / adjacent courts) are excluded.
97          """
98          # Which trajectory substrate seeds the windows (shuttle stays primary).
99          substrate: Literal["wasb", "tracknet"] = "wasb"
100         # Windowing velocity threshold for the fusion family (the engine reads
101         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
102         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
103         velocity_threshold: float = 0.02
104         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
105         # enabled ? so the default path never imports torch / touches the GPU.
106         cue_flags: dict[str, bool] = Field(default_factory=dict)
107         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
108         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
109         min_crossings: int = Field(default=0, ge=0)
110         # Served Gate B: when the person cue is on, drop windows with median in-court
players
111         # < this. 0 = OFF.
112         min_players: int = Field(default=0, ge=0)
113         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
114         court_polygon: Optional[list[list[float]]] = None
115         # Net line for the person two-sided-motion split (person features only ? the shuttle
116         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
117         net_axis: Literal["x", "y"] = "y"
118         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
119
120
```

```
121     class IndexingConfig(BaseModel):
122         default_segmenter: str = "yolo_hybrid"
123         use_gpu: bool = True
124         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
125         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
126         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
127         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
128         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
129         detector_impl: str = "auto"
130         motion_threshold: float = 0.08
131         min_segment_duration: float = 3.0
132         dead_time_merge_gap: float = 2.0
133         chunk_duration: float = 90.0
134         chunk_overlap: float = 10.0
135         detector_model: str = "fasterrcnn_resnet50_fpn"
136         detector_score_threshold: float = 0.5
137         yolo_velocity_threshold: float = 0.15
138         yolo_frame_skip: int = 3
139         yolo_tracker: str = "bytetrack.yaml"
140         yolo_prefetch_workers: int = 2
141         min_action_window_duration: float = 2.0
142         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
143         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
144         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
149         max_window_duration: float = 6.0
150         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
151         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154         # Only cuts (recall can't drop). OFF by default until measured/promoted.
155         window_hard_cap: bool = False
156         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
162         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163         window_inactivity_end: float = 1.5
164         inactivity_rally_thresh: float = 0.20
165         inactivity_min_density: float = 0.30
166         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
```

```
blob: after
167         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
172         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174         window_blob_fallback: bool = False
175         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
177         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179         window_blob_factor: float = 4.0
180         log_yolo_candidates: bool = True
181         store_yolo_candidates: bool = True
182         ai_handoff_padding: float = 2.0
183         ai_max_workers: int = 5
184         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185         # chunks of a high-bitrate (4K) source blow past the provider upload cap
186         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189         # Matches gemini_refine's --clip-scale-width default.
190         ai_chunk_scale_width: int = 1280
191         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
192         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196         skip_ai_handoff: bool = False
197         # Optional debug knob: trim every video to the first N seconds before processing.
198         debug_duration: Optional[float] = None
199
200         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202         fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204         @field_validator("default_segmenter")
205         @classmethod
206         def segmenter_must_be_registered(cls, v: str) -> str:
207             # Registry check happens at runtime in main/segmenter selection.
208             # Here we just allow any string for forward compatibility.
209             return v
210
211         @model_validator(mode="after")
212         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214             # step makes `while t < duration: t += step` loop forever, growing memory before
any
```

```
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
```

```
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]


    class StorageConfig(BaseModel):
        """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
        behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
        + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
        (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
        **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
        (boto3/google auth chains supply credentials). See ``backend/storage/``."""
        backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
        # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
        output_root: str = ""
        local: dict[str, Any] = Field(default_factory=dict)
        s3: dict[str, Any] = Field(default_factory=dict)
        gcs: dict[str, Any] = Field(default_factory=dict)
        gdrive: dict[str, Any] = Field(default_factory=dict)


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)

        # Allow extra keys during transition so existing config.json files do not break.
```

```
329         model_config = {
330             "extra": "allow",
331             "validate_assignment": True,
332         }
333 
334         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
335         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
336         # attribute access / the typed config. A nested *section* is returned as a plain
dict
337         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
338         # section model instead breaks it (a model has no .get), which bit every validator
on
339         # real runs. Scalar leaves are returned as-is.
340         def get(self, key: str, default: Any = None) -> Any:
341             _MISSING = object()
342             val: Any = _MISSING
343             if key in type(self).model_fields or hasattr(self, key):
344                 val = getattr(self, key, _MISSING)
345             if val is _MISSING:
346                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
347                 # discovered dynamically from the model fields, so a future top-level
section
348                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
349                 for fname in type(self).model_fields:
350                     section = getattr(self, fname, None)
351                     if isinstance(section, BaseModel) and hasattr(section, key):
352                         val = getattr(section, key)
353                         break
354             if val is _MISSING:
355                 return default
356             if isinstance(val, BaseModel):
357                 return val.model_dump(mode="json")
358             return val
359 
360         def __getitem__(self, key: str) -> Any:
361             val = self.get(key)
362             if val is None and key not in type(self).model_fields:
363                 # Re-raise like a real dict for unknown top-level
364                 raise KeyError(key)
365             return val
366 
367 
368     # Convenience type alias for callers that want to annotate
369     Config = AppConfig
370 
```

11. assistant (2026-06-20T09:01:21.073Z)

Good. So `config.get("indexing", {})` works on the `AppConfig` model (it has a `.get()` method and returns a plain dict for nested sections). So `_resolve_runner` will be called with a dict. And `detector_impl` would be in that dict. Confirmed.

Now let me verify the WASB hybrid and fusion_hybrid `_build_native_runner` overrides, and confirm tracknet_hybrid does NOT override `_build_native_runner`.

12. user (2026-06-20T09:01:21.863Z)

```
1       """WASB-based hybrid play segmenter.
2 
3       Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4       WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5       badminton weights and runs today, so this is the live shuttle-motion substrate.
```

```
6
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15      from typing import Any, Dict, Tuple
16
17      from backend.pipeline.segmenters.base import segmenter_registry
18      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19      from backend.pipeline.detectors.base import DetectorRunner
20      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24          family = "wasb"
25          display_label = "WASB"
26
27          @property
28          def name(self) -> str:
29              return "wasb_hybrid"
30
31          @property
32          def description(self) -> str:
33              return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                      "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41              # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42              # seam (imported only when detector_impl='native' is actually selected).
43              from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45              return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                             "detector_impl": "native", "device": cfg.device}
47
48
49      segmenter_registry.register(WasbHybridSegmenter)
50
```

13. assistant (2026-06-20T09:01:27.912Z)

Let me check tracknet_hybrid to confirm it does NOT override `_build_native_runner`, and check fusion_hybrid.

14. user (2026-06-20T09:01:28.691Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
```

```
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
```

```
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
77              # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
78              # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
79              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
80              if substrate != "wasb":
81                  raise NotImplementedError(
82                      "detector_impl='native' supports the WASB substrate only ? set "
83                      "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
84              from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
85              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
86              return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
87                                             "fusion_substrate": "wasb", "device": cfg.device}
88
89          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
90                                      frame_width: float, video_path: str,
91                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
92              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
93              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
94              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
95              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
96              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
97                                              min_crossings=0,
estimate=self._axis_estimate(points))
98              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
99
100             fcfg = idx_cfg.get("fusion", {})
101             if fcfg.get("cue_flags", {}).get("person", False):
102                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
103             return stats
104
105         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
106         # Person features (from _person_features when cue_flags.person) are now persisted
```

```
for the harness
107         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
108         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
109         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
110         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
111         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
112
113         def _axis_estimate(self, points: List[Any]) -> tuple:
114             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
115             computed once per video, not once per candidate window (it depends only on
points)."""
116             key = id(points)
117             if self._axis_cache_key != key or self._axis_cache is None:
118                 self._axis_cache_key = key
119                 self._axis_cache = rally_gate.estimate_play_axis(points)
120             est = self._axis_cache
121             assert est is not None
122             return est
123
124         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
125                              frame_width: float, video_path: str,
126                              fcfg: Dict[str, Any]) -> Dict[str, float]:
127             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
128             frame range and compute the scale/translation-invariant person features. Lazily
builds
129             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
130             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
131             from backend.pipeline.detectors.person_detector import PersonDetector
132             if self._person is None:
133                 self._person = PersonDetector(self.config)
134
135             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
136             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
137             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
138             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
139             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
140             fw = det["frame_width"] or frame_width
141             fh = det["frame_height"]
142             # If we can't trust the frame dims, every normalised feature would silently
collapse
143             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
144             if fw <= 0 or fh <= 0:
145                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
146
147             poly = fcfg.get("court_polygon")
148             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
149             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
150             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
151             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
152                                               court_polygon=polygon, net=net)
153
154         def _select_for_handoff(self, windows: List[Dict[str, Any]],
155                                 window_stats: List[Dict[str, Any]],
156                                 idx_cfg: Dict[str, Any]) -> List[int]:
157             """Served Gate A/B over the (already-enriched) per-window stats. Default
```

```
thresholds
158             (0) keep every window ? identical to the substrate. Persisted candidates are
159             untouched (full superset).
160
161             NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
162             thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
163             where no segment clears ``min_crossings``), the gate falls back to the full
ungated
164             set rather than emit a video with zero rallies. This makes Gate A/B strictly
165             *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
166             the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
167             below it always wins on an otherwise-empty video. Observed on real footage
168             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
169             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
170             fcfg = idx_cfg.get("fusion", {})
171             min_crossings = int(fcfg.get("min_crossings", 0))
172             min_players = int(fcfg.get("min_players", 0))
173             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
174
175             keep: List[int] = []
176             for i, st in enumerate(window_stats):
177                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
178                     continue
179                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
180                     continue
181                 keep.append(i)
182
183             if not keep and window_stats:
184                 # Gating emptied a non-empty video ? fall back to the ungated set.
185                 return list(range(len(window_stats)))
186             return keep
187
188
189     segmenter_registry.register(FusionSegmenter)
190
```

15. assistant (2026-06-20T09:01:31.926Z)

Confirmed. Now let me verify tracknet_hybrid does not override `_build_native_runner` (it would inherit the base raise), and check what `_resolve_train_detector` does after the build ? and how `cmd_train` handles a `None` return from `_resolve_train_detector`.

16. user (2026-06-20T09:01:32.595Z)

```
1       """TrackNet-based hybrid play segmenter.
2
3       Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4       in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5       tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6       purpose-built trajectory model localises it far more precisely, giving cleaner
7       candidate rallies than person-velocity heuristics.
8
9       The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11      TrackNet runner and identity.
12      """
13      from typing import Any, Dict, Tuple
14
15      from backend.pipeline.segmenters.base import segmenter_registry
```

```
16      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17      from backend.pipeline.detectors.base import DetectorRunner
18      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21      class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22          family = "tracknet"
23          display_label = "TrackNet"
24
25          @property
26          def name(self) -> str:
27              return "tracknet_hybrid"
28
29          @property
30          def description(self) -> str:
31              return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                      "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35              cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36              return TrackNetRunner(cfg), {
37                  "weights_path": cfg.weights_path,
38                  "queue_length": cfg.queue_length,
39              }
40
41
42      segmenter_registry.register(TrackNetHybridSegmenter)
43
```

17. assistant (2026-06-20T09:01:40.009Z)

Confirmed: `TrackNetHybridSegmenter` does NOT override `_build_native_runner`, so it inherits the base class version that raises `NotImplementedError`.

Now let me trace the full crash path. The finding claims that `worker train --trajectory-source detector --segmenter tracknet_hybrid` with `detector_impl='native'` crashes with an uncaught traceback. Let me trace:

1. `cmd_train` (line 192) ? at line 227, `if args.trajectory_source == "detector"` ? line 229 `runner = _resolve_train_detector(args, config)`.
2. `_resolve_train_detector` (line 162) ? line 183 `runner, _ = seg._resolve_runner(config.get("indexing", {}))`.
3. `_resolve_runner` ? `impl == "native"` ? `self._build_native_runner(idx_cfg)` ? raises `NotImplementedError`.

There is no try/except wrapping the call at line 183, nor at line 229 in `cmd_train`. Let me confirm there's nothing higher up that catches it.

Looking at `main()` (line 340-342): `return args.func(args)` ? no try/except. And `if __name__ == "__main__": sys.exit(main())` ? no try/except. So the `NotImplementedError` propagates all the way up and crashes with a traceback.

Now let me reproduce this directly to be certain. Let me check the tests first to understand how to invoke this, and whether the dev environment can run it.

18. user (2026-06-20T09:01:40.345Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
```

```
    on the droplet against MinIO.
    """
    import json

    import pytest

    from backend.config import load_config
    from backend.worker import __main__ as wmain


    # ---------------------------------------------------------------- pure helpers

    @pytest.mark.parametrize("raw,expected", [
        ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
    ])
    def test_coerce(raw, expected):
        assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


    def test_apply_overrides_on_typed_config():
        cfg = _fresh_config()
        wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
                                     "sport=tennis"])
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.min_segment_duration == 1.5
        assert cfg.sport == "tennis"


    def test_apply_overrides_rejects_bad_format():
        cfg = _fresh_config()
        with pytest.raises(ValueError):
            wmain._apply_overrides(cfg, ["no_equals_sign"])


    def test_write_intervals_csv(tmp_path):
        segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
                {"start_time": 9.0, "end_time": 12.5}]
        out = tmp_path / "intervals.csv"
        wmain._write_intervals_csv(segs, str(out))
        lines = out.read_text().splitlines()
        assert lines[0] == "start,end,shot_count,ending_reason"
        assert lines[1].startswith("2.000,5.000,4,")
        assert lines[2].startswith("9.000,12.500,,")


    def test_write_report_json(tmp_path):
        out = tmp_path / "report.json"
        wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
        data = json.loads(out.read_text())
        assert data["video_id"] == "vid1" and data["segment_count"] == 1
        assert data["segments"][0] == {"start": 1.0, "end": 2.0}


    def test_parser_infer_accepts_set_and_uris():
        args = wmain.build_parser().parse_args([
            "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
            "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
        assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
        assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


```

```
    # ------------------------------------------------------------- cmd_infer (local,
mocked)

    class _OKResult:
        passed = True
        validator_name = "fake"
        message = "ok"

        def model_dump(self):
            return {"validator_name": "fake", "passed": True, "message": "ok"}


    class _FailResult(_OKResult):
        passed = False
        message = "too blurry"


    class _FakeSeg:
        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return [
                {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
                {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
            ]


    def _fresh_config():
        # Load defaults without touching any cwd config.json.
        import tempfile
        p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
        p.write("{}")
        p.close()
        return load_config(p.name)


    @pytest.fixture
    def mocked_pipeline(monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_OKResult()], {}))
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


    def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "bucket"
        scratch = tmp_path / "scratch"

        rc = wmain.main([
            "infer", "--video-uri", str(video), "--out-uri", str(out),
            "--config", str(cfg), "--scratch", str(scratch),
            "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
        ])
        assert rc == 0
        intervals = out / "outputs" / "vid123" / "intervals.csv"
        report = out / "outputs" / "vid123" / "report.json"
        assert intervals.exists() and report.exists()
```

```
        rows = intervals.read_text().splitlines()
        assert len(rows) == 3                          # header + 2 rallies
        assert json.loads(report.read_text())["segment_count"] == 2


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_FailResult()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
"o"),
                         "--config", str(cfg), "--scratch", str(tmp_path / "s")])
        assert rc == 2


    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---


    def test_cmd_train_orchestration(tmp_path, monkeypatch):
        import os
        import subprocess

        labels_dir = tmp_path / "corpus" / "labels"
        labels_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
                         "--version", "0.0.1", "--config", str(cfg), "--scratch",
str(scratch)])
        assert rc == 0
        # bundle pushed to <out-uri>/weights/<version>/
        assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
        assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
        # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2
        assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


```

```
187     def test_parser_train_detector_args():
188         args = wmain.build_parser().parse_args([
189             "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
"1.0.0",
190             "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
191         assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
192
193
194     def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
195         """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
196         RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
197         import os
198         import subprocess
199
200         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
201         corpus = tmp_path / "corpus"
202         labels_dir = corpus / "labels"
203         videos_dir = corpus / "videos"
204         labels_dir.mkdir(parents=True)
205         videos_dir.mkdir(parents=True)
206         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
207         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
208         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
209         (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
210         (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
211         out_dir = tmp_path / "out"
212         scratch = tmp_path / "scratch"
213         cfg = tmp_path / "config.json"
214         cfg.write_text("{}")
215
216         def fake_run(cmd, *a, **k):
217             out = cmd[cmd.index("--out") + 1]
218             ver = cmd[cmd.index("--version") + 1]
219             bdir = os.path.join(out, ver)
220             os.makedirs(bdir, exist_ok=True)
221             for fn in ("weights.json", "manifest.json"):
222                 with open(os.path.join(bdir, fn), "w") as f:
223                     f.write("{}")
224             return subprocess.CompletedProcess(cmd, 0)
225         monkeypatch.setattr(subprocess, "run", fake_run)
226
227         rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
228                          "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
229                          "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230         assert rc == 0
231         specs = json.loads((scratch / "manifest.json").read_text())
232         assert len(specs) == 2
233         # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234         assert all(os.path.exists(s["trajectory"]) for s in specs)
235         assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub
runner's output
236
```

19. assistant (2026-06-20T09:01:44.056Z)

Now let me reproduce the crash directly to confirm the scenario described in the finding. I'll write a small reproduction that calls `_resolve_train_detector` with `segmenter='tracknet_hybrid'` and `detector_impl='native'`.

20. user (2026-06-20T09:01:54.065Z)

```
UNCAUGHT NotImplementedError: detector_impl='native' is not supported for the 'TrackNet' family
? only WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.
```

21. assistant (2026-06-20T09:01:57.298Z)

Reproduced. The `NotImplementedError` is uncaught and propagates out of `_resolve_train_detector`. Now let me confirm it propagates all the way through `cmd_train` ? `main()` ? as a real traceback (not caught anywhere). Let me reproduce the full `main()` path.

22. user (2026-06-20T09:02:00.919Z)

created

23. user (2026-06-20T09:02:12.453Z)

```
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
346, in <module>
    sys.exit(main())
             ~~~~^^
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
342, in main
    return args.func(args)
           ~~~~~~~~~^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
229, in cmd_train
    runner = _resolve_train_detector(args, config)
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
183, in _resolve_train_detector
    runner, _ = seg._resolve_runner(config.get("indexing", {}))
                ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py", line 85, in _resolve_runner
    return self._build_native_runner(idx_cfg)
           ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py", line 60, in _build_native_runner
    raise NotImplementedError(
    ...<2 lines>...
        f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
NotImplementedError: detector_impl='native' is not supported for the 'TrackNet' family ? only
WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.
EXIT CODE: 1
```

24. assistant (2026-06-20T09:02:24.258Z)

Fully reproduced through the real CLI entrypoint. The command crashes with an uncaught traceback and exit code 1, instead of returning a clean exit code 2 with an error message.

Now let me adversarially probe the finding. The finding makes some specific claims I should verify:

1. **The line number** ? finding says `@ backend/worker/__main__.py:183`. Confirmed: line 183 is `runner, _ = seg._resolve_runner(config.get("indexing", {}))`.

2. **The function's contract claims a clean error path** ? the docstring (line 165) says "Returns the runner, or prints an error + returns None". Confirmed ? the function violates its own documented contract.

3. **The seam already produces clean errors for non-trajectory segmenters** (line 180) and **failed healthchecks** (line 184-187 return None ? exit 2). Confirmed.

4. **The claimed expected behavior: "a clean exit code 2"** ? Let me verify `cmd_train` returns 2 when `_resolve_train_detector` returns None. At line 229-231: `runner = _resolve_train_detector(...); if runner is None: return 2`. Confirmed.

Now let me check the proposed fix for correctness. The finding suggests catching `NotImplementedError` only. But is `NotImplementedError` the only exception type that `_build_native_runner`/`_build_runner` can raise on misconfiguration? The fusion native branch also raises `NotImplementedError` (line 81). Let me also consider whether `seg_cls(None, config)` itself or `_build_runner` could raise other things ? but the finding's scope is specifically the `NotImplementedError` from native+unsupported-family, which is precisely what's reproduced. The proposed fix is sound for that scenario.

Let me also double-check: is there a scenario where this is unreachable in practice? The finding describes a "normal misconfiguration" ? a user picks `--segmenter tracknet_hybrid` (a valid, registered segmenter, explicitly suggested in the error message at line 181: "use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)") combined with `detector_impl='native'`. This is a plausible misconfiguration, not a contrived one ? the worker's own error message recommends `tracknet_hybrid`, and `native` is a documented detector_impl value.

Let me verify the fusion+native+tracknet-substrate variant also crashes (the finding mentions it).

25. user (2026-06-20T09:02:32.188Z)

```
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py", line 81, in _build_native_runner
    raise NotImplementedError(
        "detector_impl='native' supports the WASB substrate only ? set "
        "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
NotImplementedError: detector_impl='native' supports the WASB substrate only ? set
indexing.fusion.substrate='wasb' (no native TrackNet runner yet).
EXIT CODE: 1
```

26. assistant (2026-06-20T09:02:44.339Z)

Both variants the finding describes are confirmed: `tracknet_hybrid` + native, and `fusion_hybrid` with `substrate=tracknet` + native. Both crash with an uncaught traceback (exit 1) instead of the clean exit 2.

Now let me assess severity. The finding rates it "major". Let me consider:
- This is a worker CLI tool. A misconfiguration crashes with a traceback instead of a clean error+exit-2.
- The function explicitly documents a clean-error contract that it violates.
- The seam already produces clean errors for sibling misconfigurations (non-trajectory segmenter, failed healthcheck), so this is an inconsistency/gap, not a missing feature.
- It's a normal misconfiguration (valid registered segmenter + documented detector_impl value), not a contrived one.
- Impact: it does NOT corrupt data, produce wrong results, or affect the default/happy path. It's a poor error-handling/UX issue on a misconfig path ? an ugly traceback vs a clean message. The actual harm is limited: the user still learns the cause (the `NotImplementedError` message is descriptive and printed in the traceback), and `--trajectory-source detector` with native is itself a niche GPU-box path.

This aligns with "major" being slightly high ? it's a real bug with a clear contract violation, but the user-facing consequence is a messy traceback rather than a clean exit, and the error message is still informative. I'd put it at **minor** to lower-major. Given the contract violation and the existing clean-error siblings (which makes the gap a genuine inconsistency in a path the worker itself recommends), I'll call it **minor** ? it's a legitimate defect but the practical severity is a cosmetic/UX error-handling gap, not a correctness or data issue.

Let me verify the proposed fix is correctly scoped ? confirm `cmd_infer`/the live CLI path is unaffected (the live CLI calls `_resolve_runner` inside `process_video` which already returns `[]` cleanly on healthcheck failure, but would also crash on the native+tracknet case). That's a separate path though, outside this finding's scope. The finding is specifically about `_resolve_train_detector`.

The finding is accurate, reproducible, points to the exact line, and describes a real failure scenario with a sound fix.

27. user (2026-06-20T09:02:58.348Z)

Structured output provided successfully

28. assistant (2026-06-20T09:03:04.868Z)

I've completed the adversarial verification. The finding is **real** but I've corrected its severity from **major** to **minor**.

Summary: I reproduced the crash end-to-end through the real CLI entrypoint. `worker train --trajectory-source detector --segmenter tracknet_hybrid` with `detector_impl='native'` raises an uncaught `NotImplementedError` (exit code 1, full traceback) because `_resolve_train_detector` at `backend/worker/__main__.py:183` doesn't wrap `seg._resolve_runner(...)`, and the base `_build_native_runner` raises for any family without a native runner (tracknet_hybrid inherits the base raise; fusion_hybrid raises when substrate='tracknet'). This violates the function's own documented contract ("prints an error + returns None") and is inconsistent with its sibling misconfig paths, which cleanly return exit 2.

I downgraded to minor because the practical impact is an error-handling/UX gap (ugly traceback + exit 1 vs. clean message + exit 2) on a niche GPU-box misconfiguration path ? no data corruption, no wrong results, no effect on the default path, and the surfaced exception message is itself descriptive. The proposed fix (catch `NotImplementedError`, print, return None) is correct.